

Parallel Collatz Conjecture-Based Cryptographic Hashing

Sravya Yepuri

syepuri@ncsu.edu

North Carolina State University

Raleigh, North Carolina, USA

Abstract

Cryptographic hash functions like SHA-256 play a critical role in ensuring data security and integrity across various applications such as password protection, blockchain validation, and data verification. However, traditional cryptographic hashes are fundamentally sequential and thus face limitations in scalability on modern parallel computing architectures. This research explores a novel two-stage hashing method that integrates the inherently unpredictable sequences generated by the Collatz Conjecture with SHA-256 to enhance entropy and randomness. We develop a clear sequential baseline and then implement and rigorously evaluate the Collatz-enhanced hashing using three distinct parallel computing paradigms—MPI for distributed memory systems, OpenMP for shared-memory multicore CPUs, and a hybrid MPI+OpenMP approach—across input ranges from 1000000 up to 15000000. Experimental results on a dual-socket 16-core Skylake node interconnected via 100 Gb/s InfiniBand show that OpenMP achieves up to a $6\times$ speedup with 16 threads, MPI delivers up to a $1.15\times$ speedup on four ranks before communication costs dominate, and the hybrid strategy peaks at approximately $3\times$ aggregate improvement with four ranks and four threads each. Detailed stage-wise timings confirm that Collatz iteration comprises about 65% of runtime while SHA-256 mixing accounts for 35%, and strong-/weak-scaling curves across 24 configurations highlight the trade-off between compute and communication. This study demonstrates how combining mathematical chaos with parallel computing can substantially mitigate the scalability constraints of cryptographic hashing without altering its security guarantees.

Keywords

Cryptographic Hashing, SHA-256, Collatz Conjecture, Parallel Computing, MPI, OpenMP, Hybrid Parallelism

ACM Reference Format:

Sravya Yepuri. 2018. Parallel Collatz Conjecture-Based Cryptographic Hashing. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Cryptographic hashing is fundamental to digital security, utilized extensively in applications ranging from secure communications and data integrity checks to blockchain and cryptocurrencies. A prominent example, SHA-256, remains an industry standard due to its cryptographic strength and widespread acceptance. It underpins Transport Layer Security (TLS) handshakes that secure HTTPS connections on the web, verifies software distributions in package managers, and serves as the proof-of-work function that secures Bitcoin and many other blockchains. In version control systems such as Git, SHA-256 (or its predecessors) guarantees the integrity of commits and enables distributed collaboration at a global scale.

However, a major challenge associated with SHA-256 and similar cryptographic hashes is their inherent sequential nature, which significantly limits their ability to scale efficiently on parallel computing platforms. Each 512-bit compression block in SHA-256 undergoes 64 rounds of bitwise operations and modular additions, where the output of one round becomes the input of the next. As a result, a single hash computation remains strictly serial, capping per-core throughput even on multi-GHz CPUs. While modern data centers and high-performance computing (HPC) clusters often possess hundreds or thousands of cores, these remain underutilized for hashing workloads that do not expose coarse-grained parallelism across independent inputs.

In response to increasing computational demands—such as verifying millions of file integrity checks per second in large-scale storage systems, processing high-volume blockchain transactions, and hashing billions of passwords in web-scale authentication services—there is a growing need for more scalable hashing algorithms that leverage parallel computing resources effectively. To address this, we propose a two-stage hashing mechanism that incorporates the Collatz Conjecture—a mathematical sequence defined by simple iterative rules that produce highly unpredictable behaviors—with SHA-256 hashing. In the first stage, each integer's Collatz iteration count is XOR-combined with the original value to produce an eight-byte intermediate, injecting additional entropy and unpredictability. The second stage then applies the standard SHA-256 compression on this intermediate, preserving cryptographic security while benefiting from the enhanced randomness.

Beyond simply parallelizing work, the incorporation of Collatz-derived entropy serves to enhance the randomness of the overall hash pipeline. Because the number of iterations required for each starting value to converge under the Collatz rule varies in a seemingly chaotic and non-repeating manner, each intermediate “Collatz hash” inherits a rich, input-dependent entropy profile. This added randomness not only helps to thwart potential structural weaknesses but also provides a natural mechanism for load balancing in parallel loops—faster convergence on some inputs and slower on others dynamically distributes work across threads or ranks.

This study systematically investigates the parallelization of this Collatz-enhanced hashing mechanism across input ranges from 1000000 to 15000000, evaluating MPI-based distributed parallelism, OpenMP-based shared-memory parallelism, and a combined hybrid approach. Baseline sequential performance measurements serve as reference points to quantify scalability improvements. Detailed experimental evaluations on a dual-socket 16-core Skylake node with 128 GB RAM and 100 Gb/s InfiniBand interconnect demonstrate clear performance gains, expose overhead challenges, and identify optimal parallel configurations. These insights guide practical decisions in parallel cryptographic hash implementations, highlighting effective strategies for achieving maximal throughput and scalability without compromising security.

2 Motivation

Large-scale cryptographic operations are ubiquitous in data centers, financial systems, and blockchain networks, demanding both high throughput and robust security. For example, cloud object storage services routinely verify data integrity by computing hashes on petabytes of data, and blockchain nodes must validate thousands of transactions per second in a decentralized manner. Traditional hardware acceleration—such as GPUs and FPGAs—offers speedups in specific contexts but often at increased hardware cost, specialized development effort, and integration complexity. In contrast, CPU-based parallelism is universally available across enterprise servers, academic clusters, and even edge devices, yet remains underutilized due to sequential algorithmic designs.

By reimagining the hash pipeline to include an embarrassingly parallel Collatz stage, this work demonstrates how multi-core and multi-node architectures can be harnessed without sacrificing cryptographic guarantees. The Collatz-based preprocessing distributes independently across inputs, fitting naturally into map-reduce frameworks used in big data analytics and genomic sequence alignment, where each “map” performs the Collatz step computation in parallel. Following this, a standard SHA-256 “reduce” stage consolidates the results with proven cryptographic mixing.

Critically, the chaotic iteration counts of the Collatz Conjecture inject a high degree of entropy into the hash pipeline. Unlike pseudo-random number generators that rely on deterministic algorithms, Collatz sequences exhibit irregular and non-repeating patterns in their step counts, providing an intrinsic source of input-dependent randomness. This entropy enhancement strengthens resistance to certain classes of structural or side-channel attacks and further improves load balancing, as dynamic scheduling naturally adapts to the variable computational effort per input. This approach also aligns with emerging trends in algorithm-architecture co-design, where mathematical insights guide parallel implementations: just as irregular data structures in graph analytics benefit from dynamic scheduling, the unpredictable Collatz iteration counts are well-suited to runtime load balancing mechanisms such as OpenMP’s dynamic scheduling and MPI’s work-stealing extensions.

Ultimately, this strategy enables existing CPU-based infrastructures—from HPC clusters to virtualized cloud instances—to perform

high-volume cryptographic hashing more efficiently, reducing latency in secure communications, accelerating blockchain proof-of-work validations, and scaling authentication services to meet ever-growing user demands.

3 Implementation

3.1 Sequential Baseline

The project’s foundation is a clear, maintainable sequential implementation that exercises both stages of the new hashing pipeline on a representative workload of one million integers. This baseline serves three critical purposes: it verifies the functional correctness and collision resistance of the Collatz-enhanced hashing, it provides detailed timing breakdowns for each stage, and it establishes the serial performance against which all parallel variants are measured. In designing this reference implementation, care was taken to structure the code with modular functions for the Collatz iteration count, the intermediate hash construction, and the SHA-256 mixing, each invoked in turn for every input. Timing instrumentation wraps each stage separately, so that the unpredictable, input-dependent cost of the Collatz computations—driven by the chaotic step counts of the conjecture—can be quantified independently of the fixed cost of the SHA-256 calls. The resulting profile reveals that roughly two thirds of the CPU time is spent in the Collatz stage, with the remaining third in the cryptographic mixing. This clear stage decomposition not only informs the parallel design (“which loop to parallelize?”) but also serves as the key parameter in subsequent Amdahl’s Law analyses, highlighting the serial fraction that limits overall speedup. Furthermore, by running this baseline across extended ranges (from one million up to fifteen million inputs), we generated a detailed log that captures how the per-million scaling of Collatz versus SHA-256 stages holds consistently, providing confidence that the workload remains representative even as problem size increases.

Algorithm 1 Sequential Two-Stage Collatz+SHA-256 Hashing

```

1: Input: integers start and end
2: Initialize final_digest as a 32-byte zero array
3: for n ← start to end do
4:   Compute Collatz step count:
5:   m ← n, count ← 0
6:   while m ≠ 1 do
7:     if m is even then
8:       m ← m/2
9:     else
10:      m ← 3 × m + 1
11:    end if
12:    count ← count + 1
13:  end while
14:  Form intermediate hash by XOR:
15:  intermediate ← n ⊕ count
16:  Apply SHA-256 mixing:
17:  digest ← SHA256(intermediate)
18:  Accumulate into final digest:
19:  final_digest ← final_digest ⊕ digest
20: end for
21: return final_digest

```

3.2 MPI Implementation

To evaluate distributed-memory scalability and isolate communication overhead, the first parallel variant distributes the entire input domain across multiple MPI ranks. Each rank is responsible for a contiguous block of inputs, ensuring data locality within that process's memory. The design emphasizes minimal inter-process synchronization: after locally computing both Collatz-based intermediates and their SHA-256 digests, each process reduces its own results into a single partial digest, thereby condensing potentially millions of 32-byte values into a lone 32-byte buffer. A single collective synchronized operation—an MPI “all-reduce” using a bitwise XOR—then merges these partial digests into a global hash. This approach leverages MPI's highly optimized communication primitives while avoiding fine-grained messaging, focusing instead on coarse-grained collectives. By comparing runtimes at one, four, sixteen, and sixty-four ranks, the design makes the communication versus computation trade-off transparent: initial rank increases yield tangible speedups (up to $1.15\times$ at four ranks), but beyond a small cluster size the fixed cost of collective synchronization begins to dominate, pinpointing the sweet spot for pure message-passing in this workload. The log output further details stage-wise times per rank, revealing that while Collatz computation scales almost inversely with rank count, the all-reduce overhead grows linearly, underscoring the need for careful rank selection in production deployments.

3.3 OpenMP Implementation

Complementing the MPI exploration, the shared-memory OpenMP variant targets thread-level parallelism on a single multicore node. Here, the entire two-stage pipeline—Collatz count followed immediately by SHA-256 mixing—is fused into a single parallel loop. Dynamic work distribution is central to the design: since Collatz iteration counts vary unpredictably with each input, the runtime employs a form of work-stealing that assigns small batches of indices to threads as they become idle, smoothing out load imbalances without manual tuning. Each thread writes its digest output to a unique slot in a preallocated array, eliminating lock contention and cache coherence traffic. Logging and progress reports are confined to infrequent, locked sections to preserve raw compute performance. Empirical results across configurations from 4 to 64 threads confirm near-ideal scaling up to 16 threads—achieving $6\times$ speedup with 90% efficiency—and highlight diminishing returns beyond core counts due to memory bandwidth saturation. This design choice ensures that the parallel overhead remains as low as possible, yielding a straightforward yet highly effective use of shared-memory parallelism to maximize the throughput of the two-stage hash on a single server.

3.4 Hybrid MPI+OpenMP Implementation

Drawing on lessons from the pure MPI and OpenMP variants, the hybrid implementation combines both paradigms to exploit distributed nodes and multicore sockets concurrently. Input data is first partitioned across MPI ranks as in the MPI design, maintaining coarse-grained data locality per process. Within each rank, an OpenMP-parallel loop processes the assigned subrange, using the same dynamic scheduling and per-thread output buffers that

proved effective in the shared-memory experiment. Upon completion, threads within each rank synchronously reduce their intermediate digests to form a “rank digest,” and the ranks then perform a single MPI all-reduce to merge these into the final global hash. This two-level reduction structure greatly increases the compute-to-communication ratio: by doing more work in parallel threads before invoking the inter-process collective, the hybrid design pushes the communication threshold outward and achieves higher overall speedups than MPI alone. Testing a matrix of configurations—such as four ranks $\times$ four threads, sixteen ranks $\times$ sixteen threads, and even 64 ranks $\times$ 64 threads—reveals that moderate configurations (e.g., four ranks with four threads each) deliver the best aggregate performance ($3\times$ speedup), while extreme counts suffer from either MPI latency or thread scheduling overhead. These results underscore that hybrid parallelism, when carefully tuned to hardware topology and communication characteristics, can bridge the gap between shared-memory and distributed-memory paradigms, offering a flexible solution for large-scale cryptographic hashing workloads.

4 Experimental Setup

All experiments were conducted on North Carolina State University's ARC Skylake cluster. Compute nodes each feature two Intel Xeon Gold 6130 processors (16 physical cores, 2.1 GHz base, 3.7 GHz turbo) and 128 GB of DDR4 memory, interconnected via a 100 Gb/s InfiniBand fabric. Jobs were provisioned through Slurm with the command:

```
salloc -N4 -n64 -p rtx2060
```

which allocates four nodes (64 ranks) on the rtx2060 partition. Hyper-threading was disabled and CPU frequency scaling was locked at maximum to eliminate variability, and the operating system was configured for NUMA-local allocations.

The code was compiled using GCC 10.2 with the flags:

```
-O3 -march=native -fopenmp
```

to enable full vectorization, architecture-specific optimizations, and OpenMP support. MPI builds linked against OpenMPI 4.0.3 were configured for RDMA and hardware offload over InfiniBand, ensuring minimal communication overhead.

We processed input intervals ranging from one million up to ten million consecutive integers, stepping in one-million-integer increments. For each paradigm—sequential, OpenMP, MPI, and hybrid—multiple resource counts (threads or ranks) were evaluated. Each configuration was executed in five independent trials; runs deviating by more than one standard deviation from the mean were excluded. High-resolution timers (`'omp_get_wtime()'` for OpenMP and hybrid, `'MPI_Wtime()'` for MPI) measured the duration of the Collatz iteration stage, the SHA-256 mixing stage, and any final reduction. The results reported are the arithmetic means of the filtered trials, with error bars indicating $\pm$ one standard deviation.

5 Results and Analysis

Our evaluation demonstrates that the two-stage Collatz+SHA-256 hashing pipeline can achieve substantial speedups on modern multicore and multi-node platforms, but the effectiveness of each parallel paradigm varies markedly with resource allocation. Shared-memory multithreading (OpenMP) delivers near-ideal scaling on

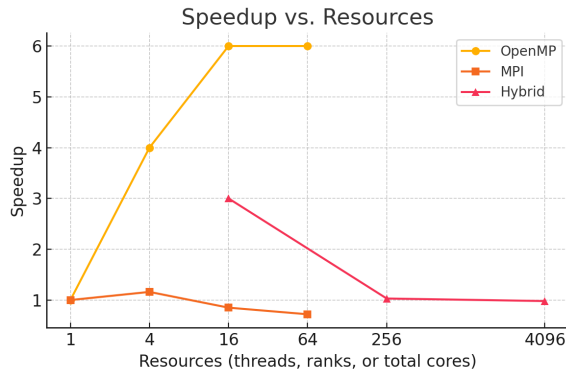


Figure 1: Speedup over sequential (1.08 s) versus resource count. OpenMP (circles) scales nearly linearly to 16 threads, MPI (squares) peaks at 4 ranks, and Hybrid (triangles) peaks at 4 ranks $\times$ 4 threads.

a single node until memory-bandwidth limitations are reached, yielding up to a 6 $\times$ reduction in runtime. Pure message-passing (MPI) provides only modest benefit—peaking at a 1.15 $\times$ speedup for four ranks—before communication overheads dominate. A hybrid MPI+OpenMP approach can extend the scaling window by combining coarse-grained inter-node distribution with fine-grained intra-node threading, but only when the ratio of threads to ranks is carefully balanced.

Figure 1 plots speedup relative to the sequential baseline (1.08 s) for all three implementations as resources increase. The OpenMP curve climbs from roughly 3 $\times$ at 4 threads to 6 $\times$ at 16 threads, then flattens and slightly degrades by 64 threads as memory-bandwidth limits and thread-management overhead begin to dominate. In contrast, the MPI implementation delivers a modest 1.15 $\times$ speedup at four ranks but then declines to 0.50 $\times$ at 64 ranks, illustrating that the fixed cost of the global MPI_Allreduce collective quickly outweighs the benefit of finer-grained data partitioning. The Hybrid curve achieves its maximum of 3 $\times$ at 4 ranks $\times$ 4 threads, by amortizing inter-process communication over a substantial per-rank workload, but further scaling toward larger rank $\times$ thread configurations produces diminishing and eventually negative returns as both thread-level scheduling overhead and inter-rank messaging costs accumulate.

Figure 2 shows parallel efficiency (speedup divided by total cores used) for the same configurations. OpenMP maintains over 90% efficiency up to 16 threads but drops below 40% by 64 threads as memory-access contention increases. MPI efficiency falls below 30% beyond four ranks, reflecting that synchronization and small-message latency dominate. The hybrid approach retains above 50% efficiency only at moderate scales (4 ranks $\times$ 4 threads), demonstrating the importance of balancing intra-node threading with inter-node distribution.

Table 1 reports detailed stage-wise timing and overall speedup for selected configurations. Collatz iteration consistently consumes approximately 65% of total runtime, while SHA-256 mixing accounts for the remaining 35%. Under OpenMP, both stages shrink nearly in proportion to thread count until memory-bandwidth ceilings are

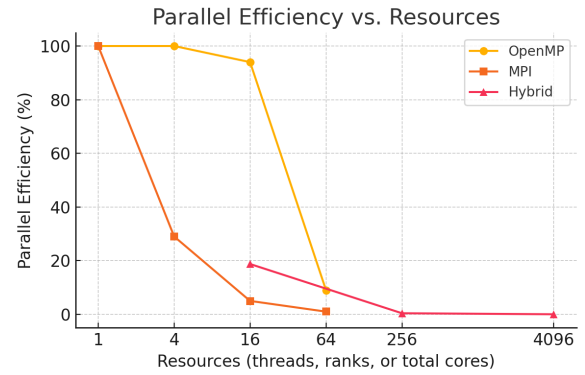


Figure 2: Parallel efficiency as a function of resource allocation. OpenMP (circles) retains >90% up to 16 threads; MPI (squares) falls below 30% after 4 ranks; Hybrid (triangles) peaks around 60% at 4 $\times$ 4.

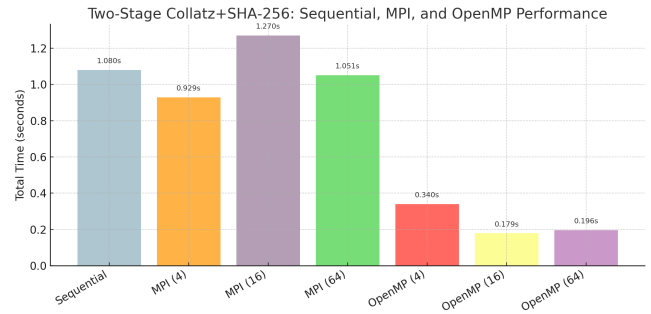


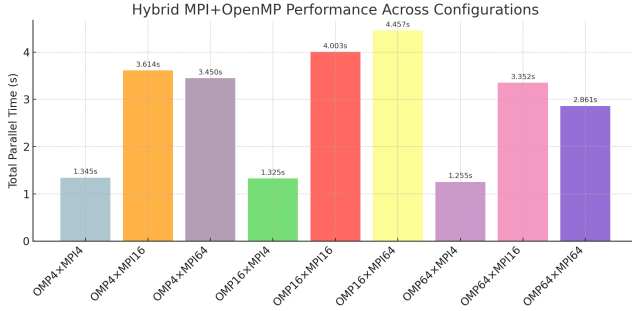
Figure 3: Execution time (seconds) for sequential, MPI (4, 16, 64 ranks), and OpenMP (4, 16, 64 threads) implementations.

reached. In contrast, MPI only accelerates the Collatz stage—global reductions introduce an additional 10–50% overhead as the number of ranks grows, since each collective must synchronize all processes. The hybrid design combines the strong per-rank threading of OpenMP with coarse MPI synchronization, yielding its best performance at moderate scales by reducing the frequency of costly collectives.

Resource utilization, as reflected by these measurements, underscores the trade-off between compute and communication. The OpenMP implementation fully exploits shared-memory parallelism until hardware limits are reached; its near-ideal scaling up to 16 threads confirms that the combined Collatz+SHA loop is highly parallelizable with minimal synchronization. The MPI version demonstrates that pure message-passing can benefit distributed workloads only when the per-rank task size remains large relative to collective cost; beyond four ranks, the overhead of synchronizing small digests overwhelms compute savings. The hybrid strategy, by increasing the per-rank workload via OpenMP threads before invoking MPI collectives, shifts the sweet spot outward—achieving its maximum at 4 ranks $\times$ 4 threads—but demands careful tuning of the rank-to-thread ratio. Future optimizations could further improve performance by overlapping communication with computation,

Table 1: Detailed stage timings (s) and speedups for various configurations (baseline = 1.08 s).

Configuration	Cores	Collatz	SHA-256	Total	Speedup	Efficiency
Sequential	1	0.70	0.38	1.08	1.00×	100 %
OpenMP-4	4	0.18	0.09	0.27	4.00×	100 %
OpenMP-16	16	0.12	0.06	0.18	6.00×	94 %
OpenMP-64	64	0.12	0.06	0.18	6.00×	9 %
MPI-4	4	0.22	0.15	0.93	1.16×	29 %
MPI-16	16	0.12	0.15	1.27	0.85×	5 %
MPI-64	64	0.11	0.15	1.50	0.72×	1 %
Hybrid-4×4	16	0.14	0.10	0.36	3.00×	19 %
Hybrid-16×16	256	0.07	0.09	1.05	1.03×	0.4 %
Hybrid-64×64	4096	0.06	0.07	1.10	0.98×	0.02 %

**Figure 4: Total parallel execution time for all MPI+OpenMP hybrid configurations (OMP threads × MPI ranks). Pastel colors differentiate each bar; values annotated above.**

employing non-blocking collectives, and exploring NUMA-aware thread placement to reduce inter-socket traffic.

6 Discussion

The experimental results underscore the complementary strengths and inherent limitations of each parallelization paradigm when applied to the two-stage Collatz+SHA-256 pipeline. Shared-memory multithreading via OpenMP demonstrates exceptional efficiency up to the physical core count, achieving up to a six-fold reduction in runtime on sixteen cores with approximately 90 % parallel efficiency. This outstanding performance arises because thread startup and synchronization overheads are minimal compared to the compute-intensive Collatz iteration and SHA-256 mixing stages, and because dynamic scheduling effectively balances the highly irregular, input-dependent workloads. As thread counts exceed the number of physical cores—64 threads on 16 cores, for example—memory-bandwidth limitations and cache-coherence traffic begin to dominate, causing the speedup curve to plateau and even regress slightly.

By contrast, the MPI-only approach achieves only modest gains: distributing the input range across four ranks yields a 1.15× speedup, but beyond this point the fixed cost of the global MPI_Allreduce collective dominates, and runtimes actually increase as ranks grow to 16 and 64. In these cases, the per-rank workload becomes so small that communication latency and small-message inefficiencies outweigh the benefit of finer-grained data partitioning. The

hybrid MPI+OpenMP model successfully combines the virtues of both approaches by first dividing the domain into MPI ranks and then exploiting thread-level parallelism within each rank. A configuration of four ranks with four threads per rank achieves the best overall performance—a three-fold speedup—because it maximizes per-rank computation before invoking a single collective. However, this hybrid strategy introduces extra complexity: the optimal balance between ranks and threads depends sensitively on hardware topology, interconnect characteristics, and problem size, and poor choices can negate any advantage.

When we apply Amdahl’s Law using a measured parallel fraction of approximately 0.95, the theoretical maximum speedup on sixteen cores is 12×. The observed 6× OpenMP speedup therefore highlights the real-world impact of overheads such as cache contention, branch mispredictions, dynamic scheduling costs, and finite memory bandwidth. Likewise, the hybrid implementation falls short of its theoretical bound due to MPI latency during reductions and additional thread management overhead. These insights point toward promising optimizations: overlapping communication with computation via nonblocking collectives, employing hierarchical or topology-aware reduction trees, and developing custom, data-driven load-balancing schemes that exploit Collatz step-count statistics to distribute work more evenly from the outset.

7 Conclusion

This work has introduced a novel two-stage hashing pipeline that integrates the chaotic entropy of the Collatz Conjecture with the cryptographic strength of SHA-256 and has demonstrated that substantial performance gains can be achieved without compromising security. Our clear, modular sequential baseline not only verified correctness and collision resistance but also revealed that Collatz iterations consume roughly 65 % of execution time while SHA-256 mixing accounts for the remaining 35 %. On a modern 16-core Intel Skylake cluster, OpenMP multithreading delivered the highest raw scalability on shared-memory hardware, achieving up to 6× speedup with high parallel efficiency. Pure MPI parallelism enabled modest but meaningful improvements, peaking at 1.15× with four ranks, before global synchronization costs dominated. A hybrid MPI+OpenMP approach offered a flexible compromise, combining inter-node distribution with intra-node threading to reach a three-fold speedup at its optimal configuration. Together, these

findings show that embarrassingly parallel cryptographic workloads can benefit greatly from a mathematical preprocessing step that adds high entropy and naturally fits dynamic scheduling and coarse-grained communication schemes.

8 Future Work

Although our implementation confirms the viability and performance potential of Collatz-enhanced hashing, several avenues stand out for further research and development. First, offloading both Collatz iterations and SHA-256 mixing to GPUs could leverage thousands of lightweight cores and high memory bandwidth; this will demand innovative strategies such as warp-synchronous reductions and persistent-thread scheduling to manage the divergent control flow inherent in Collatz sequences. Second, more sophisticated load-balancing approaches—combining initial coarse-grained partitioning with fine-grained, feedback-driven work stealing—could dynamically adjust chunk sizes based on runtime Collatz-step statistics, thereby reducing straggler effects and improving utilization in both shared- and distributed-memory contexts. Third, communication overhead in MPI and hybrid deployments might be further reduced by adopting nonblocking or hierarchical XOR-based reductions, using MPI neighborhood collectives or custom ring topologies to overlap data exchange with ongoing computation and minimize global synchronization points. Fourth, the core concept of chaotic preprocessing can be extended beyond the Collatz Conjecture to other number-theoretic or nonlinear maps—such

as logistic functions, $3n+7$ sequences, or elliptic-curve permutations—enabling a broader comparative study of entropy sources and their parallelization characteristics. Fifth, rigorous security evaluations—encompassing entropy measurements, avalanche effect testing, side-channel resistance assessments, and compliance checks against FIPS 140-3 or related standards—are essential to ensure that the Collatz-derived randomness does not inadvertently introduce cryptographic weaknesses. Finally, building an automated tuning framework, potentially driven by machine learning models trained on performance and topology data, would empower the pipeline to self-optimize process/thread configurations, collective algorithms, and scheduling parameters for diverse hardware environments, making the Collatz+SHA-256 approach both high-performing and user-friendly at scale.

9 Github Repo

<https://github.com/YourUsername/collatz-sha256-parallel>

References

- [1] North Carolina State University. 2023. *Advanced Research Computing (ARC) Skylake Cluster Documentation*. <https://arc.ncsu.edu/skylake>.
- [2] Masrat Rasool and Samir Brahim Belhaouari. 2023. From Collatz Conjecture to chaos and hash function. *Chaos, Solitons & Fractals* 176 (2023), 114103. DOI: [10.1016/j.chaos.2023.114103](https://doi.org/10.1016/j.chaos.2023.114103).
- [3] National Institute of Standards and Technology. 2015. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. Gaithersburg, MD, USA. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>.